Project reference

Implementation, reproducibility and evidence guide

Prakhar Gupta | Section F | Roll No. 2430010465
Aditya Dubey | Section F | Roll No. 2430010453

Public project: https://github.com/ZekRock18/brain-tumor-segmentation

Read this package in order

Start with START_HERE.html, then the teacher report. For a visual demonstration open results/report.html. For execution evidence open results/executed.ipynb. For a new Kaggle run use the clean notebook and metadata under notebooks/.

Location	Contents and purpose
Three root PDFs	Teacher report; this technical reference; presentation script and viva notes
results/	Executed notebook, real metrics, all figures/GIFs, HTML report, logs and completion metadata
notebooks/	Clean inference notebook and Kaggle kernel metadata
weights/	Both original pretrained checkpoint binaries and provenance
reproducibility/	Notebook builder, validation helpers, PDF/package builder and upstream source snapshot
CONTENTS.csv / MANIFEST_SHA256.csv	File inventory and integrity hashes

Included and deliberately excluded

The delivery includes both trained weight artifacts and the source needed for the inference workflow. It excludes credentials, raw MRI scans, abandoned runs, synthetic demo results, Git history, caches and the original author's example GIFs. Upstream training code is retained as reference; it was not run for these results. The original source snapshot excludes empty output placeholders.

Data and preprocessing

Input suffix	Meaning	Model channel
t1n	T1-weighted MRI	0
t1c	Contrast-enhanced T1 MRI	1
t2w	T2-weighted MRI	2
t2f	FLAIR MRI	3
seg	Reference label volume	Evaluation target only

Discovery and selection

The notebook recursively locates attached data, groups the four modalities and segmentation by patient identity, and retains complete cases. The saved manifest fixes the evaluated patient set: 00047-000, 00068-000, 00111-000 and 00170-000 (all prefixed BraTS-SSA-). Seed 42 supports repeatability, but the manifest is the definitive record if dataset versions change.

Normalization

For each modality, take voxels with intensity greater than zero, clip intensities to the 0.5th and 99.5th percentiles of that mask, subtract the clipped masked mean and divide by the clipped masked standard deviation plus epsilon. Reset background to zero. There is no per-case model training.

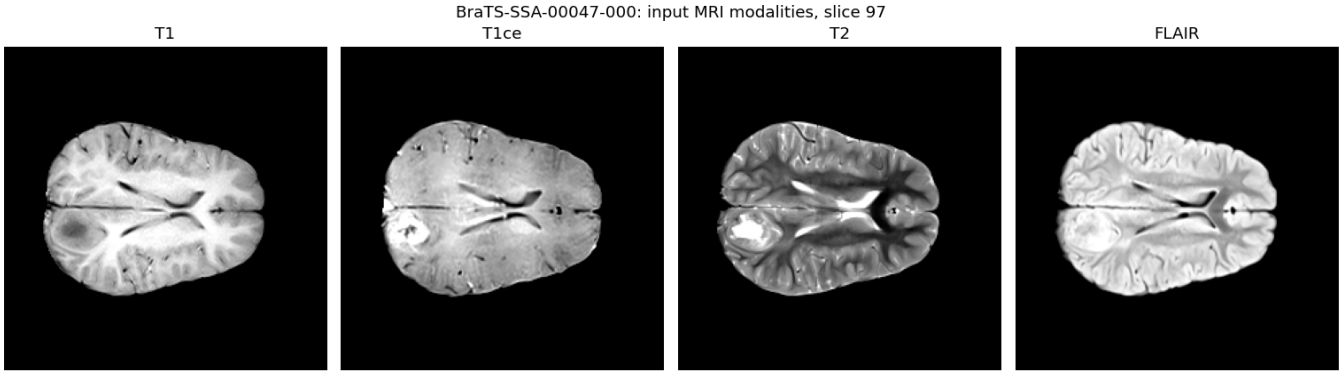
Targets and nested regions

The implemented conversion maps 3 to 4 and defines ET = {4}, TC = {1,4}, WT = {1,2,4}. These are overlapping evaluation regions rather than three mutually exclusive classes. The display paints WT first, then TC, then ET so the red enhancing region remains visible. Independently verifying the dataset's label semantics remains an explicit limitation of this delivery.

Geometry and visual selection

The sampled volumes use the full 240 × 240 × 155 grid. Header spacing is passed to the HD95 computation. The report uses an axial slice with maximal ground-truth WT area for illustration; this does not change which voxels are scored. Four modality preview images and animations provide additional views.

Understand the inputs visually

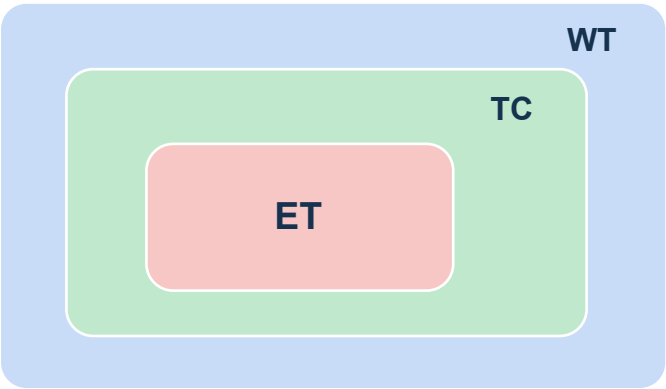


A slice, a voxel and a volume

A pixel is one location in a 2D image. A voxel is one location in a 3D image grid. An axial slice is one 2D plane through the volume. A 240 × 240 × 155 scan contains 8,928,000 voxels. Four modality volumes are stacked as channels; they describe the same grid with different intensity contrasts.

What normalization changes

MRI brightness scales can differ. Normalization changes the input intensity distribution while preserving its spatial grid. As a simple arithmetic example, a clipped brain intensity of 140 with masked mean 100 and standard deviation 20 becomes (140 - 100) / 20 = 2. This example is explanatory; it is not a measured intensity from the shown patient.



ET is inside TC.

TC is inside WT.

The regions overlap;

they are scored separately.

Schematic of overlapping region definitions; not a medical image or measured tumor shape. In the overlays, the visible colors are drawn in layers, so the blue portion is WT outside TC and the green portion is TC outside ET.

Why three channels instead of one class

The models output a probability-like sigmoid value for each region at each voxel. A voxel can belong to ET, TC and WT simultaneously. Each channel is thresholded independently; the shown pipeline does not add an explicit anatomical nesting correction after thresholding.

Models and inference

Feature	UNet3D	UNETR
Core design	3D convolutional encoder-decoder; skip connections	Transformer encoder; CNN decoder and skip connections
Normalization	GroupNorm	LayerNorm encoder; GroupNorm in decoder
Parameter count	19,992,603	19,327,907
Input / output	4 MRI channels / 3 region logits	4 MRI channels / 3 region logits

Checkpoint loading

Construct each architecture from its checkpoint configuration and load the state dictionary with strict=True. This rejects missing or unexpected parameter keys. SHA-256 checks identify the exact binary files. Parameter counts alone do not prove the weights are pretrained, and model runtime is not evidence of checkpoint authenticity.

Sliding-window prediction

Set model.eval(), disable gradients, use 128-cubed windows with 25% overlap and batch size one, apply sigmoid to each window output, and blend overlapping probability predictions using Gaussian weights. Threshold at 0.5 to form masks. MONAI is used when available; a tested built-in implementation handles its absence, including padding and cropping.

GPU and reproducibility

The recorded run used a Tesla T4, torch 2.11.0+cu128 and Python 3.13.15. The notebook requires CUDA by default to prevent an accidental slow CPU run. Two visible T4 devices do not imply that both were used: this workflow uses the selected CUDA device and does not implement multi-GPU parallelism. Seed 42 is fixed, but hardware and dependency changes can still affect numerical reproducibility.

Timings are descriptive

The first UNet3D case took 52.54 seconds, while subsequent cases took about 9.6–10.0 seconds. UNETR cases took about 2.2 seconds. Startup effects were not isolated in a controlled benchmark, so these measurements should not be promoted as a general speedup claim. Peak VRAM was not measured.

Understand the model workflow



What UNet3D learns to do

The encoder applies small 3D convolution filters to learn local patterns and progressively reduce spatial resolution. The decoder upsamples features to recover a mask at the input resolution. Skip connections bring earlier spatial features into the decoder, helping preserve detail. These feature weights were learned by the original author before this demonstration.

What UNETR changes

The transformer encoder represents image patches as tokens and uses attention to combine information across tokens. The convolutional decoder converts those representations back into dense voxel predictions. This is the repository's custom UNETR implementation, not a claim that any MONAI UNETR checkpoint is interchangeable with it.

Why strict loading and eval mode matter

A checkpoint stores learned tensors. The architecture defines where those tensors belong. Strict loading checks structural compatibility, while a SHA-256 hash verifies file identity. model.eval() changes training-dependent behavior such as dropout. Disabling gradients avoids building a training graph. Neither step trains the model.

Why patches overlap

A model sees less surrounding context near patch edges. Overlap lets several windows contribute to the same location, and Gaussian weighting gives more influence to the window interior. The combined probability map is cropped back to the original image shape and thresholded before scoring.

Metrics and interpretation

Dice coefficient

Dice = 2 × |prediction intersection reference| / (|prediction| + |reference|). It quantifies overlap for each of ET, TC and WT. Higher is better, with 1 indicating perfect overlap. The implementation treats two empty masks as Dice 1. A case's mean Dice averages its three region scores; the report averages those case values across four patients.

95th-percentile Hausdorff distance

HD95 is the 95th percentile of the pooled bidirectional distances between predicted and reference surfaces. Header voxel spacing gives physical distances in millimetres. Lower is better. Two empty masks produce zero. For only one empty mask, the repository uses a fixed nominal-volume diagonal penalty of about 373.13 mm; that special-case penalty is not adapted to each volume's spacing.

Model	ET HD95	TC HD95	WT HD95	Mean mm
UNet3D	5.6758	10.7677	2.6346	6.3594
UNETR	2.4415	8.6011	5.8798	5.6408

Why both metrics are needed

Overlap can be high even when a small misplaced region is far away. For example, case 00170-000 has UNet3D ET Dice 0.9668 but ET HD95 13.63 mm. A good Dice score therefore does not guarantee closely aligned boundaries. Review both masks and metrics.

Valid claims

The run demonstrates successful pretrained inference and evaluation on four labeled MRI volumes. It does not establish clinical usefulness, statistical superiority, robustness across populations, or independently verified out-of-distribution performance. No training/validation split was needed for this inference-only demo; this does not prove the cases were absent from the author's training data.

Original author results are separate

Checkpoint validation scores of 0.8694 (UNet3D) and 0.8362 (UNETR) describe the original model artifacts. They are not measurements from this four-case run. Likewise, any training or benchmark tables in the upstream README remain the original author's results.

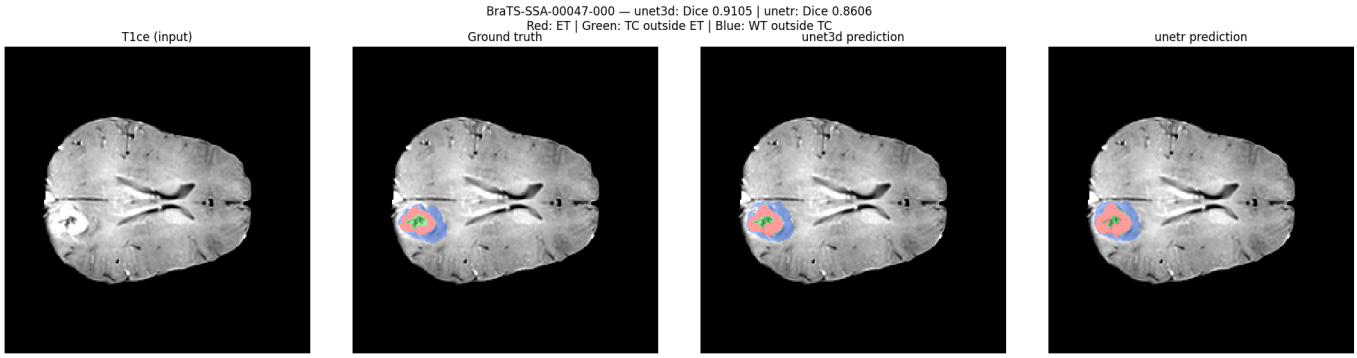
Worked example: reading a result

First understand Dice with simple numbers

Suppose a prediction contains 100 tumor voxels, the reference contains 120, and 90 are shared. Dice is 2 × 90 / (100 + 120) = 0.8182. This is an illustrative calculation only. Dice is overlap, not classification accuracy and not a probability that the patient has cancer.

Then read an actual patient

For BraTS-SSA-00047-000, UNet3D has ET Dice 0.8971, TC Dice 0.9381 and WT Dice 0.8964. Their arithmetic mean is 0.9105. The corresponding mean HD95 is 1.93 mm. Those values come from the full volume, although the figure shows one slice.



Inspect agreement and disagreement

Compare each prediction with the reference on the same slice. Look for missing colored regions, extra colored regions and boundary differences. A single matching slice does not prove good performance across the volume; use the GIF and the full-volume metrics together. Do not infer a clinical diagnosis from the overlay.

Understand averaging

The report treats each of the four patients equally. It averages the three region scores within each patient and then averages across patients. It does not pool all tumor voxels into one global Dice calculation. Large and small tumors can therefore contribute equally to the reported patient-average score.

Separate completion from scientific validation

A success marker and executed cells establish that the pipeline completed. They do not establish correct medical semantics, absence of training-data overlap, or clinical reliability. Those questions require additional dataset documentation and study design.

How to rerun on Kaggle

1. Open or import the notebook

Open the public Kaggle notebook using the link listed on the next page, or import notebooks/brain-tumor-seg-brats-africa.ipynb into your own Kaggle account. The saved executed notebook remains an offline backup of version 6.

2. Attach the inputs

Attach yakunokamizen/brats-africa and zekrock/brats-weights. Access to the latter may depend on the account. If unavailable, create a private checkpoint input from the two binaries in weights/, retaining their names and attribution. Do not upload the entire project folder as data.

3. Select a GPU session

Complete account phone verification where requested. Select GPU T4 in the notebook accelerator settings and start the session. Confirm the setup cell reports CUDA and a GPU name. An API request for a GPU is not proof of allocation; the executed device output is the evidence.

4. Run every cell in order

Keep both model and dataset inputs attached. The notebook loads weights, discovers cases, predicts and exports results. The default GPU guard should remain enabled. No training is required. Dependencies and dataset availability can change; inspect any setup failure before continuing.

5. Save the completed version

Use Save Version / Run All to preserve execution outputs. Check all code cells have run, no errors remain and END-TO-END COMPLETE appears. Confirm SUCCESS.json and all 19 listed artifacts exist. Download outputs and the executed notebook; never interpret a running status as completed results.

Local verification of this delivery

From reproducibility/, run: `python verify_kaggle_results.py ../results`. It checks patient/model coverage, finite metrics, expected hashes, readable images, multiframe GIFs, completed notebook cells and the success marker. It requires numpy, pandas and Pillow. No MRI download is needed.

Provenance and maintenance

Source snapshot

Upstream commit: a76973db5d3c4a437146716bad1236ce400d3c96. The source snapshot under reproducibility/repo retains the original framework for traceability. The notebook builder inlines its architecture, preprocessing and metrics code and adds the Kaggle workflow. The final results are from version 6, not the earlier CPU or synthetic runs.

Checkpoint identities

Full hashes and parameter counts are recorded in weights/provenance.json and results/run_metadata.json. MANIFEST_SHA256.csv covers the packaged files. The manifest excludes itself; CONTENTS.csv supplies a readable inventory. This is an integrity record, not a cryptographic signature from the original author.

Regenerating documents and code

From reproducibility/, run `python build_final_delivery.py --documents-only` to rebuild the PDFs and start page from saved metrics. This needs pandas, Pillow and ReportLab. Run `python build_kaggle_notebook.py` to regenerate the self-contained notebook from the included source. Validation helpers exercise inference/export behavior with synthetic fixtures; those fixtures are not the reported patient results.

Troubleshooting

Missing data: check attached inputs and folder discovery. CUDA guard failure: check account verification, GPU allocation and available quota in the UI. Missing MONAI: the fallback is available, but other imports must still succeed. Strict-load error: check architecture configuration and checkpoint hashes; do not suppress the error. Missing final artifacts: review the execution log and rerun from the failed cell or clean session.

Our project on GitHub — Prakhar Gupta / ZekRock18
https://github.com/ZekRock18/brain-tumor-segmentation

Public Kaggle notebook (packaged results: version 6)
https://www.kaggle.com/code/zekrock/brain-tumor-seg-brats-africa

Original model repository
https://github.com/MohanadMahran/Brain_Tumor_Benchmark

UNet3D checkpoint
https://huggingface.co/MohanadKombar/brain-tumor-unet3d-brats

UNETR checkpoint
https://huggingface.co/MohanadKombar/brain-tumor-unetr-brats

BraTS-Africa input dataset
https://www.kaggle.com/datasets/yakunokamizen/brats-africa

Attached checkpoint dataset
https://www.kaggle.com/datasets/zekrock/brats-weights

No new license is asserted for the upstream code, checkpoints or dataset. Source URLs are attribution and access references; availability and access policies may change.